CCPS LAB EXAM

Complete Arduino → Node-RED → Dashboard Guide

A practical step-by-step guide for the Node-RED questions in your CCPS question bank. It starts from wiring and Arduino code, then covers the USB/serial connection, Node-RED setup, dashboard creation, testing, and how to adapt the same flow for common exam variations.

Assumptions: Arduino UNO connected to a Windows computer by USB; a DHT11 temperature/humidity sensor and an HC-SR04 ultrasonic sensor are used in the combined example. If your exam question uses only one sensor, use the relevant part. Pin choices below are examples—match them to your wiring.

Safety: HC-SR04 Echo is a 5 V signal. Do not connect it directly to a Raspberry Pi GPIO. For Arduino UNO, the 5 V Echo signal is normally compatible. DHT11 modules may have different pin labels; follow the module's printed labels.

1. Overall connection: what talks to what?

Hardware path: Sensor → Arduino UNO → USB cable → computer running Node-RED → Serial In node → JSON node → Function node → dashboard widgets.

Arduino reads sensors and sends one JSON line through USB serial, for example:

```
{"temperature":27.5,"humidity":62,"distance":18.4}\n
```

Node-RED reads the serial line, converts JSON text into an object, runs the decision logic, and displays the result on a dashboard.

2. Wiring the sensors to Arduino UNO

DHT11 module (temperature + humidity)

DHT11 pin	Arduino UNO connection
VCC / +	5V (or module-rated supply)
GND / -	GND
DATA / S	Digital pin 2 (D2)
HC-SR04 pin	Arduino UNO connection
VCC	5V
GND	GND
TRIG	Digital pin 9 (D9)
ECHO	Digital pin 10 (D10)

If the exam uses only DHT11, you can omit the HC-SR04 wiring and distance field. If it uses only ultrasonic distance, omit the DHT11 library and its wiring.

3. Arduino IDE: install, paste, upload

1. Install Arduino IDE and open it. Connect the UNO to the computer with a USB data cable.
2. Open Tools → Board and choose Arduino Uno.
3. Open Tools → Port and select the COM port that appears for the UNO.
4. Install the DHT library: Sketch → Include Library → Manage Libraries; search for **DHT sensor library** by Adafruit and install it. If prompted, install Adafruit Unified Sensor too.
5. Create a new sketch, replace its contents with the code below, then click Verify (✓).
6. Click Upload (→). Open Tools → Serial Monitor and set baud rate to **9600** to check the JSON output. **Close Serial Monitor before using Node-RED**, because only one program can usually use the COM port at a time.

Full combined Arduino sketch

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11
#define TRIG_PIN 9
#define ECHO_PIN 10

DHT dht(DHTPIN, DHTTYPE);

float readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000UL);
  if (duration == 0) return -1; // timeout
  return duration * 0.0343 / 2.0;
}

void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}
```

```
void loop() {
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();
  float distance = readDistanceCm();

  // Send one JSON object per line for Node-RED
  Serial.print("{\"temperature\":");
  if (isnan(temperature)) Serial.print("null");
  else Serial.print(temperature, 1);

  Serial.print(",\"humidity\":");
  if (isnan(humidity)) Serial.print("null");
  else Serial.print(humidity, 1);

  Serial.print(",\"distance\":");
  if (distance < 0) Serial.print("null");
  else Serial.print(distance, 1);

  Serial.println("}");
  delay(2000);
}
```

Expected Serial Monitor output: one line every two seconds, such as {"temperature":27.5,"humidity":62.0,"distance":18.4}. If DHT or ultrasonic reading fails, that field is sent as null.

4. Start Node-RED from zero on Windows

1. Install Node.js LTS from nodejs.org/en/download if it is not already installed. Restart Command Prompt after installation.
2. Open Command Prompt and run node -v, then npm -v. Both should show version numbers.
3. Install Node-RED once with npm install -g node-red.
4. Start Node-RED by running node-red. Keep this Command Prompt window open.
5. Open your browser and go to http://localhost:1880. This is the Node-RED editor, not the dashboard.

5. Build the serial input flow — exact nodes and connections

Required order: serial in → json → debug first. Verify data before adding decision logic or dashboard widgets.

1. In the left palette, search for **serial in** and drag it onto the workspace. If missing, open Menu ■ → Manage palette → Install, search for node-red-node-serialport, and install it.
2. Double-click **serial in**. Click the pencil/edit button beside Serial Port. Select the Arduino COM port (for example COM3; use the actual port shown in Arduino IDE). Set baud rate to **9600**. Set output to a string and line delimiter to newline if the node offers that option. Save/Done.
3. Drag a **json** node and connect the output of **serial in** to its input.
4. Drag a **debug** node and connect **json** → **debug**. Set Debug to show msg.payload.
5. Click **Deploy**. Open the Debug sidebar (bug icon). You should see an object with temperature, humidity, and distance fields.

If the Serial In node is red or no data appears: close Arduino Serial Monitor; confirm correct COM port and baud rate 9600; confirm the sketch was uploaded; check that Arduino prints one JSON object per line; then Deploy again.

6. Install and open the dashboard

This guide uses the classic node-red-dashboard widgets, such as ui_text and ui_gauge, because they are commonly used in introductory lab exercises. If your lab machine already uses a different dashboard package, widget names may differ.

1. In Node-RED editor, Menu ■ → Manage palette → Install.
2. Search node-red-dashboard and click Install.
3. After installation, search the left palette for ui_text, ui_gauge, and optionally ui_chart.
4. Dashboard page URL is http://localhost:1880/ui. It may show nothing until you add widgets, assign them to a Tab and Group, and Deploy.

7. Dashboard example A — show temperature, humidity, distance

Flow: serial in → json → three Change nodes (or one Function with three outputs) → temperature ui_gauge, humidity ui_gauge, distance ui_gauge. Also connect the JSON node to Debug while testing.

Easy beginner setup: create three separate Function nodes from the JSON node. Each Function keeps only the value needed by its widget:

```
// Function: Temperature
msg.payload = Number(msg.payload.temperature);
return msg;

// Function: Humidity
msg.payload = Number(msg.payload.humidity);
return msg;

// Function: Distance
msg.payload = Number(msg.payload.distance);
return msg;
```

For each widget: double-click it → choose/create a Tab and Group → set label (Temperature / Humidity / Distance) → set units (°C / % / cm) → choose a suitable range (e.g. temperature 0–50, humidity 0–100, distance 0–400) → Done. Connect each Function to its matching widget. Click Deploy and open http://localhost:1880/ui.

Important: a Gauge expects a number. If the sensor value is null, Number(null) becomes 0; for a robust exam answer, check for invalid values before sending to a gauge. Use a separate ui_text for status messages.

8. Dashboard example B — occupancy: distance < 25 cm

Question: Show “Student Detected” if distance is below 25 cm, otherwise “No Student”.

Exact connection: serial in → json → Function → ui_text. Keep a Debug node connected to the JSON node if you need to inspect raw values.

```
let d = Number(msg.payload.distance);

if (!Number.isFinite(d)) {
    msg.payload = "Sensor Failure";
} else if (d < 25) {
    msg.payload = "Student Detected";
} else {
    msg.payload = "No Student";
}
return msg;
```

Configure ui_text: label = Occupancy; value format = {{msg.payload}}; choose a dashboard Tab and Group. Deploy. Place an object within 25 cm of the ultrasonic sensor to test the first status.

9. Dashboard example C — temperature OR humidity alert

Question: Alert if temperature > 30°C OR humidity > 75%.

```
let t = Number(msg.payload.temperature);
let h = Number(msg.payload.humidity);

if (!Number.isFinite(t) || !Number.isFinite(h)) {
    msg.payload = "Sensor Failure";
} else if (t > 30 || h > 75) {
    msg.payload = "UNCOMFORTABLE";
} else {
    msg.payload = "Normal";
}
return msg;
```

Flow: serial in → json → Function → ui_text. Set ui_text label to Classroom Alert. Remember: OR is ||; AND is &&.

10. More exam logic — reuse the same sensor connection

You do not need to rebuild the serial connection for each question. Keep **serial in** → **json**, then change the Function node logic and dashboard output to match the question.

Three-level temperature: below 25 = Comfortable; 25–30 inclusive = Normal; above 30 = Hot.

```
let t = Number(msg.payload.temperature);
if (!Number.isFinite(t)) msg.payload = "Sensor Failure";
else if (t < 25) msg.payload = "Comfortable";
else if (t <= 30) msg.payload = "Normal";
else msg.payload = "Hot";
return msg;
```

Safety distance: below 20 = Danger; 20–50 inclusive = Warning; above 50 = Safe.

```
let d = Number(msg.payload.distance);
if (!Number.isFinite(d)) msg.payload = "Sensor Failure";
else if (d < 20) msg.payload = "DANGER";
else if (d <= 50) msg.payload = "WARNING";
else msg.payload = "SAFE";
return msg;
```

Emergency: temperature > 32 AND distance < 20 at the same time.

```
let t = Number(msg.payload.temperature);
let d = Number(msg.payload.distance);
if (!Number.isFinite(t) || !Number.isFinite(d))
    msg.payload = "Sensor Failure";
else if (t > 32 && d < 20)
    msg.payload = "Emergency Condition";
else
    msg.payload = "Normal";
return msg;
```

Ventilation: temperature > 30 AND humidity > 70 → ON; otherwise OFF.

```
let t = Number(msg.payload.temperature);
let h = Number(msg.payload.humidity);
if (!Number.isFinite(t) || !Number.isFinite(h))
    msg.payload = "Sensor Failure";
else if (t > 30 && h > 70)
    msg.payload = "Ventilation ON";
else
    msg.payload = "Ventilation OFF";
return msg;
```

Sensor failure: Arduino sends null if DHT reading fails; Node-RED should not display it as a valid reading. In a Function node use msg.payload.temperature == null and the same for humidity, or use Number.isFinite after conversion.

Dynamic threshold (dashboard input)

Add a **ui_numeric** input widget labelled Distance Threshold and set an initial value (e.g. 20). Connect it to a Function node that stores the value in flow context, or use a simple flow with a Change node. Because the input and sensor arrive as separate messages, they must be stored and compared in the same Function. A common pattern is:

```
// Use one Function node with 2 outputs is not needed here;
// easiest is to store threshold from a dedicated input branch:
flow.set("threshold", Number(msg.payload));
return null;

// In the sensor branch Function:
let threshold = Number(flow.get("threshold") ?? 20);
let d = Number(msg.payload.distance);
msg.payload = (Number.isFinite(d) && d < threshold)
    ? "Unsafe Distance" : "Safe";
return msg;
```

For an exam, explain that the ui_numeric branch stores the latest threshold in flow context, and the sensor branch reads it for each new distance packet. Ensure the two branches use separate Function nodes.

11. Testing checklist and common problems

No data in Debug: verify the Arduino Serial Monitor shows JSON; close Serial Monitor; verify COM port; use 9600 baud in both Arduino and Node-RED; ensure serial in is wired to json and Debug; click Deploy.
JSON node errors: Arduino must send valid JSON, with double quotes around field names and a newline after each object. Example: {"temperature":27.5,"humidity":62,"distance":18.4}.
Dashboard page is blank: ensure a ui widget is actually connected; configure Tab and Group; click Deploy; open http://localhost:1880/ui.
Gauge shows 0 or odd value: ensure the Function sends a number, not text; check field names match Arduino JSON exactly.
Port busy: close Arduino Serial Monitor and any other program using the COM port.
Node-RED command not found: install Node.js LTS, reopen Command Prompt, then run npm install -g node-red.

12. Exam-ready one-minute explanation

“The sensors are connected to Arduino. Arduino reads temperature, humidity and distance and sends a JSON object every two seconds over USB serial at 9600 baud. In Node-RED, the Serial In node receives each line, the JSON node parses it into an object, and a Function node applies the required threshold logic. The result goes to a dashboard ui_text widget, while numeric sensor values go to gauges. I deploy the flow and open localhost:1880/ui to demonstrate the live output.”

13. Quick block wiring reference

Purpose	Node order
Inspect raw values	serial in → json → debug
Occupancy status	serial in → json → Function → ui_text
Temp/humidity alert	serial in → json → Function → ui_text
Three sensor gauges	serial in → json → 3 × Function → 3 × ui_gauge
MQTT instead of USB serial	mqtt in → json (if JSON text) → Function → ui_text/gauge

Final reminder: Always test the actual serial connection before writing the decision Function. If the question gives different thresholds, change only the numbers and preserve the correct AND/OR operator.